TEORIA DE DIAGRAMAS

Cristian Andrés Galeano Poveda

Universidad de Cundinamarca
Extensión Chía

INGENIERIA DE SOFTWARE

William Matallana

2026-I

1. Teoría de los Diagramas de Clases UML

Los diagramas de clases son un tipo de diagrama del Lenguaje Unificado de Modelado (UML) que representan la estructura estática de un sistema. Estos diagramas muestran las clases del sistema, sus atributos, métodos y las relaciones que existen entre ellas. Son fundamentales en el desarrollo de software orientado a objetos, ya que permiten visualizar el diseño antes de la implementación.

1. ¿Qué es un diagrama de clases?

Un diagrama de clases describe los diferentes tipos de objetos que existen en un sistema y cómo se relacionan entre sí. Cada clase se representa mediante un rectángulo dividido en tres secciones: el nombre de la clase, los atributos y los métodos.

2. Elementos de un diagrama de clases

• Clase: Representa un conjunto de objetos con características y comportamientos comunes.
• Atributos: Son las propiedades o características de la clase.
• Métodos: Son las acciones o comportamientos que puede realizar la clase.

3. Tipos de relaciones en diagramas de clases

3.1 Asociación

La asociación es una relación estructural que indica que una clase se comunica o interactúa con otra. Puede ser unidireccional o bidireccional y suele incluir multiplicidad (1, 0..1, *, 1..*).

3.2 Agregación

La agregación es una forma especial de asociación que representa una relación 'tiene un', donde las clases pueden existir de manera independiente. Se representa con un rombo blanco.

3.3 Composición

La composición es una relación fuerte de dependencia. Si el objeto contenedor se destruye, los objetos contenidos también. Se representa con un rombo negro.

3.4 Herencia (Generalización)

La herencia permite que una clase herede atributos y métodos de otra clase. Facilita la reutilización del código y la organización jerárquica. Se representa con una flecha con triángulo blanco.

3.5 Dependencia

La dependencia indica que una clase utiliza a otra de manera temporal. Un cambio en la clase utilizada puede afectar a la clase dependiente. Se representa con una flecha discontinua.

4. Importancia de los diagramas de clases

Los diagramas de clases permiten planificar el diseño del software, mejorar la comprensión del sistema, facilitar la comunicación entre desarrolladores y servir como base para la implementación en lenguajes de programación orientados a objetos.

2. Diagrama de casos de uso

Un diagrama de casos de uso UML es la forma principal de representar los requisitos del sistema/software para un nuevo programa de software en desarrollo. Los casos de uso especifican el comportamiento esperado (qué), no el método exacto para lograrlo (cómo). Una vez especificados, los casos de uso pueden representarse tanto textualmente como visualmente (es decir, un diagrama de casos de uso). Un concepto clave del modelado de casos de uso es que nos ayuda a diseñar un sistema desde la perspectiva del usuario final. Es una técnica eficaz para comunicar el comportamiento del sistema en términos del usuario, especificando todo el comportamiento visible externamente del sistema.

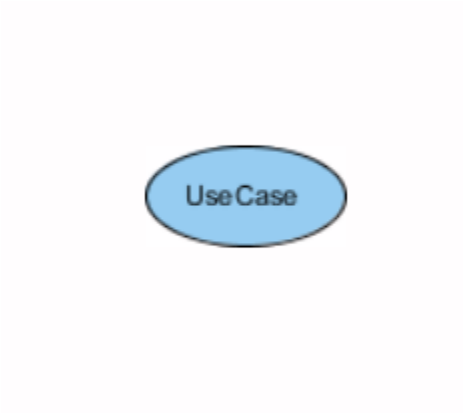
Los diagramas de casos de uso generalmente se desarrollan en la etapa inicial del desarrollo y las personas a menudo aplican el modelado de casos de uso para los siguientes propósitos:

- Especificar el contexto de un sistema
- Capturar los requisitos de un sistema
- Validar una arquitectura de sistemas
- Impulsar la implementación y generar casos de prueba
- Desarrollado por analistas junto con expertos del dominio.

Actor

- Alguien interactúa con un caso de uso (función del sistema).
- Nombrado por sustantivo.
- El actor desempeña un papel en el negocio.
- Similar al concepto de usuario, pero un usuario puede desempeñar diferentes roles.
- Por ejemplo:
    - Un profesor puede ser instructor y también investigador.
    - juega 2 roles con dos sistemas
- El actor activa casos de uso.
- El actor tiene una responsabilidad hacia el sistema (entradas) y tiene expectativas del sistema (salidas).

Caso de uso

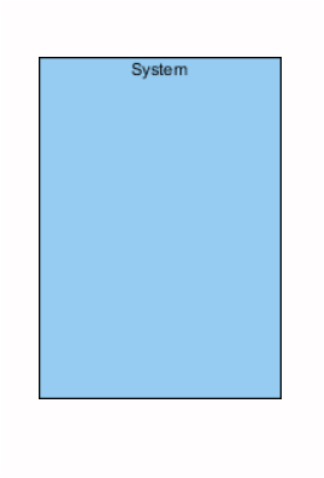


- Función del sistema (proceso: automatizado o manual)
- Nombrado por verbo + sustantivo (o frase nominal).
- es decir Haz algo
- Cada actor debe estar vinculado a un caso de uso, mientras que algunos casos de uso pueden no estar vinculados a actores.

Enlace de comunicación

- La participación de un actor en un caso de uso se muestra conectando un actor a un caso de uso mediante un enlace sólido.
- Los actores pueden estar conectados a casos de uso mediante asociaciones, lo que indica que el actor y el caso de uso se comunican entre sí mediante mensajes.

Límite del sistema



- El límite del sistema es potencialmente todo el sistema tal como se define en el documento de requisitos.
- Para sistemas grandes y complejos, cada módulo puede ser el límite del sistema.
- Por ejemplo, para un sistema ERP de una organización, cada uno de los módulos como personal, nómina, contabilidad, etc.
- puede formar un límite del sistema para casos de uso específicos de cada una de estas funciones comerciales.
- El sistema completo puede abarcar todos estos módulos que representan el límite general del sistema.

EJEMPLO

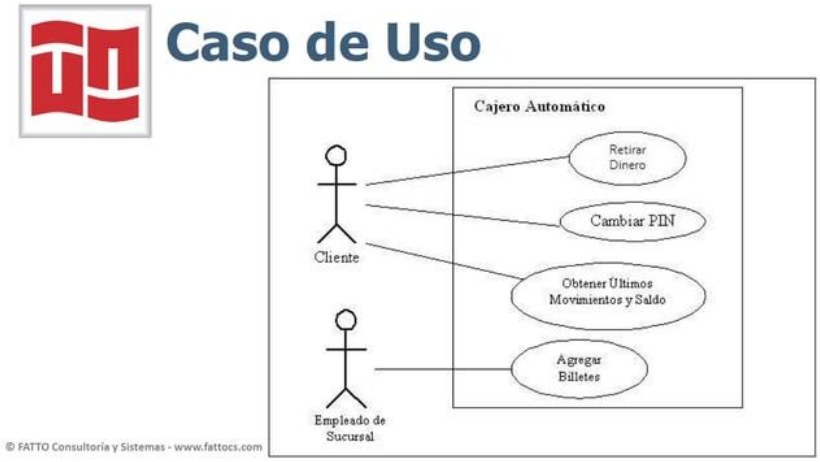


DIAGRAMA DE SECUENCIA

Son diagramas de interacción que detallan cómo se realizan las operaciones. Capturan la interacción entre objetos en el contexto de una colaboración. Se centran en el tiempo y muestran visualmente el orden de la interacción utilizando el eje vertical del diagrama para representar el tiempo, los mensajes y cuándo se envían.

Los diagramas de secuencia capturan:

- la interacción que tiene lugar en una colaboración que realiza un caso de uso o una operación (diagramas de instancia o diagramas genéricos)
- interacciones de alto nivel entre el usuario del sistema y el sistema, entre el sistema y otros sistemas, o entre subsistemas (a veces conocidos como diagramas de secuencia del sistema)

Propósito del diagrama de secuencia

- Modelar la interacción de alto nivel entre objetos activos en un sistema
- Modelar la interacción entre instancias de objetos dentro de una colaboración que realiza un caso de uso
- Modelar la interacción entre objetos dentro de una colaboración que realiza una operación
- Modele interacciones genéricas (mostrando todas las rutas posibles a través de la interacción) o instancias específicas de una interacción (mostrando solo una ruta a través de la interacción)

DIAGRAMAS DE ESTADO

Muestran los diferentes estados de una entidad. Los diagramas de máquina de estados también pueden mostrar cómo una entidad responde a diversos eventos cambiando de un estado a otro. Un diagrama de máquina de estados es un diagrama UML que se utiliza para modelar la naturaleza dinámica de un sistema.

Los diagramas de máquina de estados se utilizan habitualmente para describir el comportamiento dependiente del estado de un objeto. Un objeto responde de forma

diferente al mismo evento según su estado . Los diagramas de máquina de estados se suelen aplicar a objetos, pero también pueden aplicarse a cualquier elemento que tenga comportamiento con otras entidades, como actores, casos de uso, métodos, subsistemas, etc., y suelen utilizarse junto con diagramas de interacción (normalmente diagramas de secuencia).

¿Qué es un Estado?

Rumbaugh define que:

Un estado es una abstracción de los valores de los atributos y vínculos de un objeto. Los conjuntos de valores se agrupan en un estado según las propiedades que afectan el comportamiento general del objeto.

Características de las notaciones de máquinas de estados

Existen varias características de los Estados en general, independientemente de su tipo:

- Un estado ocupa un intervalo de tiempo.
- Un estado a menudo se asocia con una abstracción de valores de atributos de una entidad que satisface alguna(s) condición(es).
- Una entidad cambia su estado no sólo como consecuencia directa de la entrada actual, sino que también depende de algún historial pasado de sus entradas.

FORMATO DE REQUERIMIENTOS DEL SISTEMA
Ingeniería de Software I

1. Información general del proyecto

Nombre del proyecto	Sistema de Gestión de Citas
Integrantes	Cristian Andres Galeano Poveda
Programa académico	Ingeniería de Sistemas y Computación
Fecha	18/02/2026
Lenguaje de programación	Java
Tipo de aplicación	Consola

2. Descripción general del sistema

- El sistema permite gestionar las citas medicas de un consultorio mediante la aplicación de consola.
- El usuario puede registrar citas médicas, listar citas medicas, cambiar el estado de una cita, y eliminar citas.
- Cada cita con un identificador único, el nombre del paciente, la fecha de la cita y su cita.

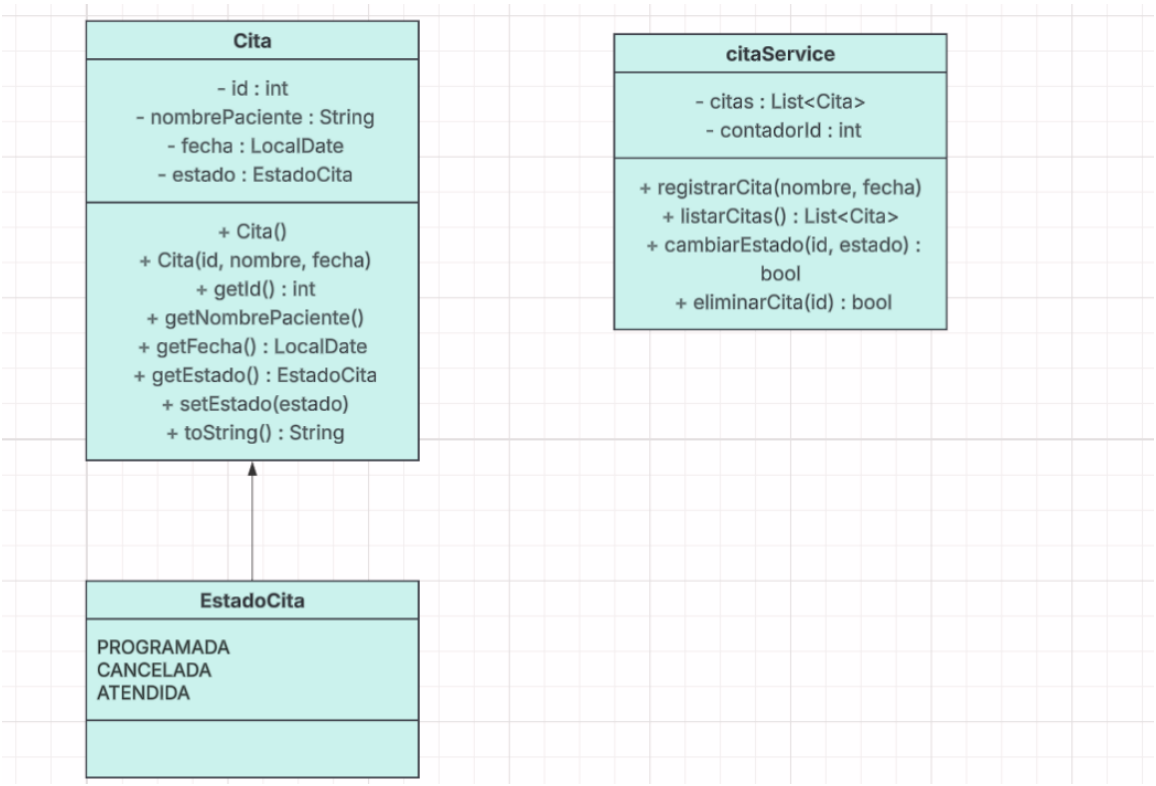
3. Requerimientos Funcionales (RF)

ID	Nombre	Descripción	Entrada(s)	Proceso	Salida
RF-01	Registrar cita medica	Permite registrar una nueva cita	Nombre del paciente, fecha	Asigna ID, y estado PROGRAMADA	Cita registrada
RF-02	Listar citas medicas	Permite visualizar todas las citas registros		Recorre la lista de las citas	Lista de citas
RF-03	Cambiar estado de cita	Permite modificar el estado de la cita	Id de la cita, nuevo estado	Valida reglas y cambia el estado	Estado actualizado
RF-04	Eliminar cita	Permite eliminar cita	Id de la cita	Busca y elimina la cita	Cita eliminada

4. Requerimientos No Funcionales (RNF)

ID	Tipo	Descripción
RNF-01	Validación	Una cita cancelada no puede cambiarse a atendida
RNF-02	Validación	El identificador de la cita debe generarse automáticamente
RNF-03	Estructura	El sistema debe estar organizado por capas: modelo, servicio y presentacion
RNF-04	Calidad	El sistema debe aplicar encapsulamiento y uso de enumeraciones (enum)

5. Relación Requerimiento – POO

ID Requerimiento	Clase	Método	Tipo
RF-01	CitaService	registrarCita()	Funcional
RF-02	CitaService	listarCita()	Funcional
RF-03	CitaService	cambiarCita()	Funcional
RF-04	CitaService	eliminarCita()	Funcional
RNF-01	CitaService	cambiarCita()	No Funcional

6. Diagrama UML